

Assignment 1: Neural Networks — IMDB Sentiment Classification

Shivam Shekher

September 26, 2026

Contents

1 Overview	2
2 Setup	2
3 Data	3
3.1 Loading the IMDB dataset	3
3.2 Preprocessing	3
3.3 Dataset summary	4
4 Model and Experiment Functions	5
4.1 Model builder	5
4.2 Experiment runner	6
5 Running the Experiments	7
6 Number of Hidden Layers (Question 1)	8
7 Number of Hidden Units (Question 2)	9
8 MSE versus Binary Crossentropy (Question 3)	10
9 tanh versus ReLU (Question 4)	11
10 Regularization to Improve Validation (Question 5)	12
11 Summary of All Experiments	13
12 Final Model	15
13 Conclusions	16

1 Overview

This report extends the IMDB sentiment-classification network from class. The baseline is a fully connected network with two hidden layers of 16 ReLU units, a sigmoid output, RMSprop, and binary crossentropy. Starting from that baseline, one factor is changed at a time so that every difference in performance can be attributed to that factor.

Assignment question	What is changed	Models compared
1. Number of hidden layers	1, 2, 3 hidden layers	1x16, 2x16, 3x16
2. Number of hidden units	8, 16, 32, 64 units per layer	2x8, 2x16, 2x32, 2x64
3. Loss function	MSE instead of binary crossentropy	2x16 BCE vs 2x16 MSE
4. Activation function	tanh instead of ReLU	16 and 32 units, ReLU vs tanh
5. Improving validation	Dropout, L2 regularization, both	16, 32 and 64 units

Evaluation protocol. Every model is trained for 20 epochs on 15,000 reviews and monitored on a 10,000-review validation set. For each model the report records the best validation accuracy and the epoch it occurred at. The weights from that best epoch are restored and scored on the 25,000-review test set, so test accuracy reflects the model at its best point rather than after it has overfit. The final model is chosen on validation accuracy only; the test set is never used for selection.

2 Setup

```
library(keras3)
library(dplyr)
library(tidyr)
library(tibble)
library(readr)
library(ggplot2)
library(scales)

SEED <- 64061L
NUM_WORDS <- 10000L # vocabulary size
VALIDATION_N <- 10000L # reviews held out for validation
BATCH_SIZE <- 512L
EXPERIMENT_EPOCHS <- 20L

set.seed(SEED)
dir.create("results", showWarnings = FALSE)
```

3 Data

3.1 Loading the IMDB dataset

```
imdb <- dataset_imdb(num_words = NUM_WORDS)

train_data  <- imdb$train$x
train_labels <- as.numeric(imdb$train$y)
test_data   <- imdb$test$x
test_labels  <- as.numeric(imdb$test$y)

cat("Training reviews:", length(train_data), "\n")
```

```
#> Training reviews: 25000
```

```
cat("Test reviews:      ", length(test_data), "\n")
```

```
#> Test reviews:      25000
```

3.2 Preprocessing

Each review is a sequence of word indices. It is converted into a multi-hot vector of length 10,000, where position j is 1 if word j appears in the review. The first 10,000 training reviews form the validation set and the remaining 15,000 are used for training during the experiments.

Memory. Stored as ordinary R numbers (8 bytes each), the four matrices would take about 6 GB, and each call to `fit()` would convert them into new Python arrays again. To avoid that, the matrices are built once, directly as 32-bit NumPy arrays (about 2 GB in total). The validation and training splits are views of the full training array, not copies, and the same arrays are reused by every model.

```
# Small Python helpers: build a multi-hot matrix and split rows without copying
reticulate::py_run_string("
import numpy as np

def multi_hot(rows, cols, n_rows, dim):
    x = np.zeros((int(n_rows), int(dim)), dtype='float32')
    x[np.asarray(rows, dtype='int64'), np.asarray(cols, dtype='int64')] = 1.0
    return x

def head_rows(x, n):
    return x[:int(n)]

def tail_rows(x, n):
    return x[int(n):]

")
py_main <- reticulate::import_main(convert = FALSE) # keep results in Python
```

```

vectorize_sequences <- function(sequences, dimension = NUM_WORDS) {
  rows <- rep(seq_along(sequences) - 1L, lengths(sequences)) # 0-based rows
  cols <- unlist(sequences, use.names = FALSE) - 1L           # word j -> col j-1
  keep <- cols >= 0L & cols < dimension
  py_main$multi_hot(rows[keep], cols[keep], length(sequences), dimension)
}

x_train_full <- vectorize_sequences(train_data)
x_test       <- vectorize_sequences(test_data)
y_train_full <- train_labels
y_test       <- test_labels

x_val          <- py_main$head_rows(x_train_full, VALIDATION_N)
partial_x_train <- py_main$tail_rows(x_train_full, VALIDATION_N)
val_idx        <- seq_len(VALIDATION_N)
y_val          <- y_train_full[val_idx]
partial_y_train <- y_train_full[-val_idx]

rm(imdb, train_data, test_data)
invisible(gc())

print(x_train_full$shape)

```

```
#> (25000, 10000)
```

```
print(x_test$shape)
```

```
#> (25000, 10000)
```

3.3 Dataset summary

```

dataset_summary <- tibble(
  Split      = c("Training", "Validation", "Test"),
  Reviews    = c(length(partial_y_train), length(y_val), length(y_test)),
  Features    = NUM_WORDS,
  `Positive rate` = c(mean(partial_y_train), mean(y_val), mean(y_test))
)
write_csv(dataset_summary, "results/dataset_summary.csv")

dataset_summary |>
  mutate(`Positive rate` = percent(`Positive rate`, accuracy = 0.1)) |>
  knitr::kable(caption = "Size and class balance of each split",
               booktabs = TRUE, format.args = list(big.mark = ","))

```

Table 2: Size and class balance of each split

Split	Reviews	Features	Positive rate
Training	15,000	10,000	49.6%
Validation	10,000	10,000	50.5%
Test	25,000	10,000	50.0%

All three splits are close to 50% positive, so accuracy is a fair metric and a model that always predicts one class would score about 50%.

4 Model and Experiment Functions

4.1 Model builder

`build_imdb_model()` builds the class architecture and exposes each factor the assignment asks about as an argument. When dropout is used, a dropout layer follows every hidden layer; when L2 is used, it is applied to every hidden layer's weights.

```
build_imdb_model <- function(hidden_layers = 2L,
                             units        = 16L,
                             activation    = "relu",
                             loss_function = "binary_crossentropy",
                             dropout_rate  = 0,
                             l2           = 0) {
  model <- keras_model_sequential(input_shape = NUM_WORDS)

  for (i in seq_len(hidden_layers)) {
    model <- model |>
      layer_dense(
        units        = units,
        activation    = activation,
        kernel_regularizer = if (l2 > 0) regularizer_l2(l2) else NULL
      )
    if (dropout_rate > 0) {
      model <- model |> layer_dropout(rate = dropout_rate)
    }
  }

  model <- model |> layer_dense(units = 1, activation = "sigmoid")

  model |> compile(
    optimizer = optimizer_rmsprop(),
    loss      = loss_function,
    metrics   = "accuracy"
  )
  model
}
```

```
}
```

4.2 Experiment runner

`run_experiment()` trains one configuration and returns its per-epoch history and a one-row summary. A checkpoint callback saves the weights from the epoch with the highest validation accuracy, and those weights are restored before the test-set evaluation.

```
run_experiment <- function(id, label, group, hidden_layers, units,
                           activation, loss_function, dropout_rate, l2) {
  set_random_seed(SEED) # same initial conditions for every model

  model <- build_imdb_model(hidden_layers, units, activation,
                           loss_function, dropout_rate, l2)

  checkpoint_path <- file.path(tempdir(), paste0(id, ".weights.h5"))

  history <- model |> fit(
    partial_x_train, partial_y_train,
    epochs          = EXPERIMENT_EPOCHS,
    batch_size      = BATCH_SIZE,
    validation_data = list(x_val, y_val),
    callbacks       = list(callback_model_checkpoint(
      filepath      = checkpoint_path,
      monitor       = "val_accuracy",
      save_best_only = TRUE,
      save_weights_only = TRUE
    )),
    verbose = 0
  )

  # keras3's fit() returns an R object; per-epoch values are in $metrics
  m <- lapply(history$metrics, function(v) as.numeric(unlist(v)))
  stopifnot(length(m$val_accuracy) == EXPERIMENT_EPOCHS)

  history_df <- tibble(
    id          = id,
    epoch       = seq_along(m$accuracy),
    train_accuracy = m$accuracy,
    val_accuracy  = m$val_accuracy,
    train_loss    = m$loss,
    val_loss      = m$val_loss
  )
  write_csv(history_df, file.path("results", paste0("history_", id, ".csv")))

  test_last <- model |> evaluate(x_test, y_test, verbose = 0)
  model |> load_model_weights(checkpoint_path)
  test_best <- model |> evaluate(x_test, y_test, verbose = 0)
```

```

best_epoch <- which.max(m$val_accuracy)

summary_row <- tibble(
  id, label, group, hidden_layers, units, activation,
  loss_function, dropout_rate, l2,
  best_epoch,
  best_val_accuracy = m$val_accuracy[best_epoch],
  last_val_accuracy = tail(m$val_accuracy, 1),
  overfit_gap       = tail(m$accuracy, 1) - tail(m$val_accuracy, 1),
  test_accuracy_best = as.numeric(test_best[["accuracy"]]),
  test_accuracy_last = as.numeric(test_last[["accuracy"]])
)
stopifnot(nrow(summary_row) == 1)

list(summary = summary_row, history = history_df)
}

```

5 Running the Experiments

```

experiment_grid <- tribble(
  ~id, ~label, ~group,
  ~hidden_layers, ~units, ~activation, ~loss_function, ~dropout_rate, ~l2,
  "base_2x16", "2x16 ReLU, BCE (baseline)", "Baseline",
  2L, 16L, "relu", "binary_crossentropy", 0, 0,
  "depth_1x16", "1x16 ReLU, BCE", "Depth",
  1L, 16L, "relu", "binary_crossentropy", 0, 0,
  "depth_3x16", "3x16 ReLU, BCE", "Depth",
  3L, 16L, "relu", "binary_crossentropy", 0, 0,
  "width_2x8", "2x8 ReLU, BCE", "Width",
  2L, 8L, "relu", "binary_crossentropy", 0, 0,
  "width_2x32", "2x32 ReLU, BCE", "Width",
  2L, 32L, "relu", "binary_crossentropy", 0, 0,
  "width_2x64", "2x64 ReLU, BCE", "Width",
  2L, 64L, "relu", "binary_crossentropy", 0, 0,
  "loss_mse_2x16", "2x16 ReLU, MSE", "Loss",
  2L, 16L, "relu", "mse", 0, 0,
  "act_tanh_2x16", "2x16 tanh, BCE", "Activation",
  2L, 16L, "tanh", "binary_crossentropy", 0, 0,
  "act_tanh_2x32", "2x32 tanh, BCE", "Activation",
  2L, 32L, "tanh", "binary_crossentropy", 0, 0,
  "reg_drop_2x16", "2x16 ReLU, dropout 0.5", "Regularization",
  2L, 16L, "relu", "binary_crossentropy", 0.5, 0,
  "reg_l2_2x16", "2x16 ReLU, L2 0.001", "Regularization",
  2L, 16L, "relu", "binary_crossentropy", 0, 0.001,
  "reg_drop_2x32", "2x32 ReLU, dropout 0.5", "Regularization",

```

```

2L, 32L, "relu", "binary_crossentropy", 0.5, 0,
"reg_drop_2x64", "2x64 ReLU, dropout 0.5", "Regularization",
2L, 64L, "relu", "binary_crossentropy", 0.5, 0,
"reg_drop_l2_2x64", "2x64 ReLU, dropout 0.5 + L2", "Regularization",
2L, 64L, "relu", "binary_crossentropy", 0.5, 0.001
)

runs <- lapply(seq_len(nrow(experiment_grid)), function(i) {
  do.call(run_experiment, as.list(experiment_grid[i, ]))
})

results_summary <- bind_rows(lapply(runs, `[`, "summary"))
histories <- bind_rows(lapply(runs, `[`, "history")) |>
  left_join(select(results_summary, id, label), by = "id")

write_csv(results_summary, "results/results_summary.csv")
write_csv(histories, "results/all_histories.csv")

cat("Experiments completed:", nrow(results_summary), "of",
    nrow(experiment_grid), "\n")

```

```
#> Experiments completed: 14 of 14
```

The tables in the next five sections use these columns:

- **Best ep. and Val. (best):** the epoch with the highest validation accuracy, and that accuracy.
- **Test (best):** test accuracy using the weights from the best epoch.
- **Test (ep. 20):** test accuracy after all 20 epochs, which shows the cost of overtraining.
- **Gap (ep. 20):** training accuracy minus validation accuracy at epoch 20. Larger values mean more overfitting.

6 Number of Hidden Layers (Question 1)

```

depth_ids <- c("depth_1x16", "base_2x16", "depth_3x16")
results_table(depth_ids, "Effect of the number of hidden layers (16 units)")

```

Table 3: Effect of the number of hidden layers (16 units)

Model	Best ep.	Val. (best)	Test (best)	Test (ep. 20)	Gap (ep. 20)
1x16 ReLU, BCE	4	89.04%	88.32%	86.38%	11.93%
2x16 ReLU, BCE (baseline)	4	88.95%	88.21%	85.84%	12.79%
3x16 ReLU, BCE	3	88.78%	88.02%	85.82%	12.88%

```
plot_curves(depth_ids, "Hidden-layer depth")
```

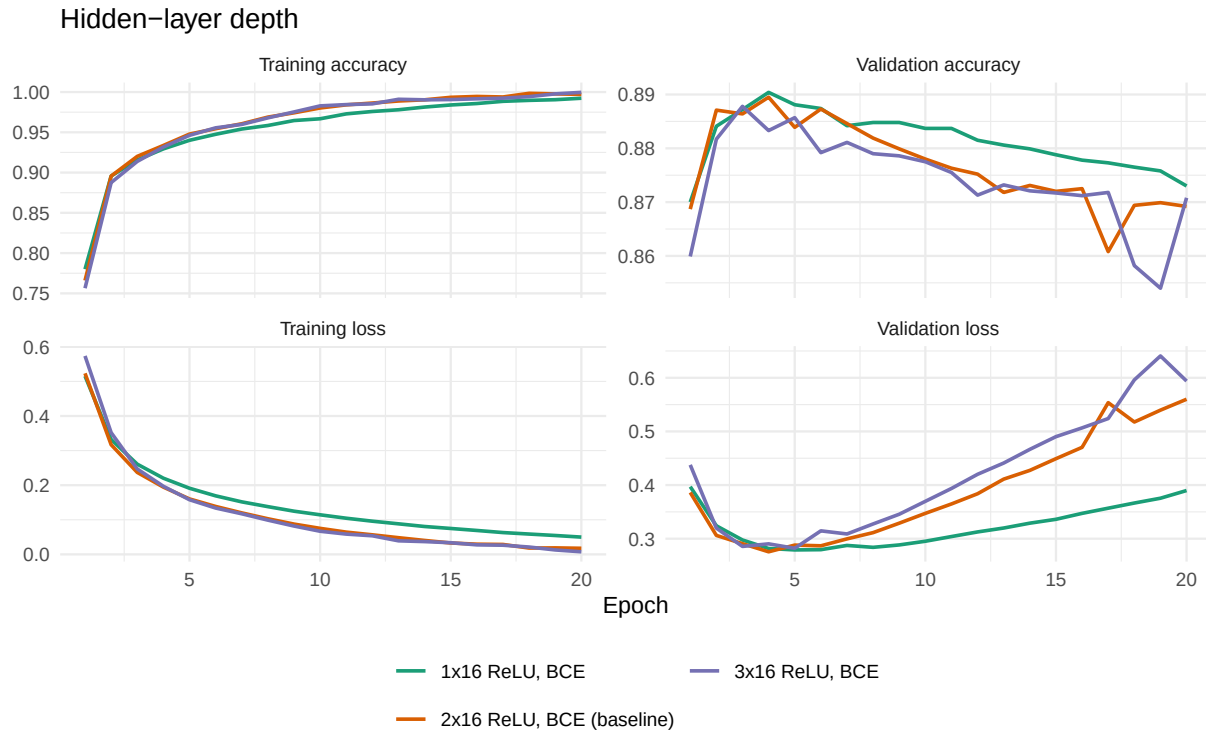


Figure 1: Training and validation curves for 1, 2 and 3 hidden layers.

7 Number of Hidden Units (Question 2)

```
units_ids <- c("width_2x8", "base_2x16", "width_2x32", "width_2x64")
results_table(units_ids, "Effect of the number of units per layer (2 layers)")
```

Table 4: Effect of the number of units per layer (2 layers)

Model	Best ep.	Val. (best)	Test (best)	Test (ep. 20)	Gap (ep. 20)
2x8 ReLU, BCE	4	89.03%	88.11%	85.90%	12.77%
2x16 ReLU, BCE (baseline)	4	88.95%	88.21%	85.84%	12.79%
2x32 ReLU, BCE	4	88.81%	88.25%	86.20%	12.82%
2x64 ReLU, BCE	2	89.26%	88.24%	86.55%	12.55%

```
plot_curves(units_ids, "Hidden units per layer")
```

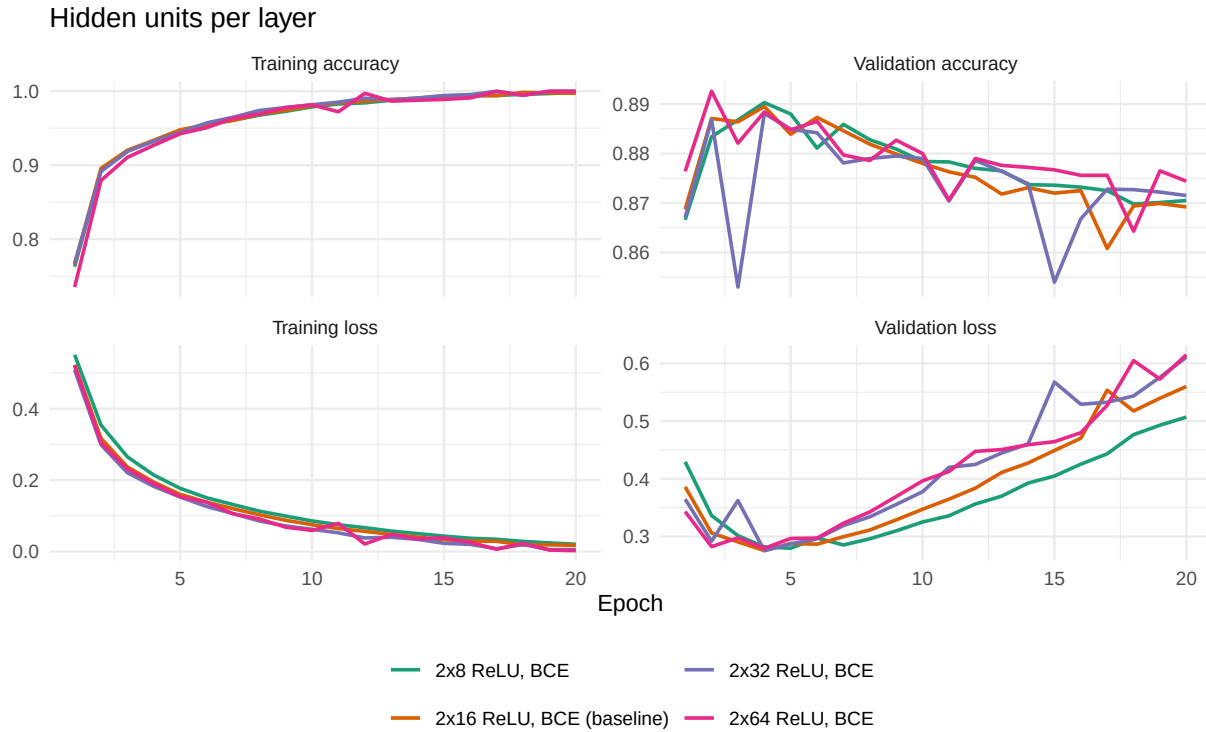


Figure 2: Training and validation curves for 8, 16, 32 and 64 units per layer.

8 MSE versus Binary Crossentropy (Question 3)

```
loss_ids <- c("base_2x16", "loss_mse_2x16")
results_table(loss_ids, "Binary crossentropy versus mean squared error")
```

Table 5: Binary crossentropy versus mean squared error

Model	Best ep.	Val. (best)	Test (best)	Test (ep. 20)	Gap (ep. 20)
2x16 ReLU, BCE (baseline)	4	88.95%	88.21%	85.84%	12.79%
2x16 ReLU, MSE	4	89.01%	88.34%	86.26%	11.69%

```
plot_curves(loss_ids, "Loss function")
```



Figure 3: Training and validation curves for BCE and MSE. The loss panels are on different scales, so compare accuracy rather than loss values.

9 tanh versus ReLU (Question 4)

```
activation_ids <- c("base_2x16", "act_tanh_2x16", "width_2x32", "act_tanh_2x32")
results_table(activation_ids, "ReLU versus tanh at 16 and 32 units")
```

Table 6: ReLU versus tanh at 16 and 32 units

Model	Best ep.	Val. (best)	Test (best)	Test (ep. 20)	Gap (ep. 20)
2x16 ReLU, BCE (baseline)	4	88.95%	88.21%	85.84%	12.79%
2x16 tanh, BCE	2	88.92%	88.18%	85.27%	12.92%
2x32 ReLU, BCE	4	88.81%	88.25%	86.20%	12.82%
2x32 tanh, BCE	2	88.83%	88.21%	85.20%	13.31%

```
plot_curves(activation_ids, "Activation function")
```

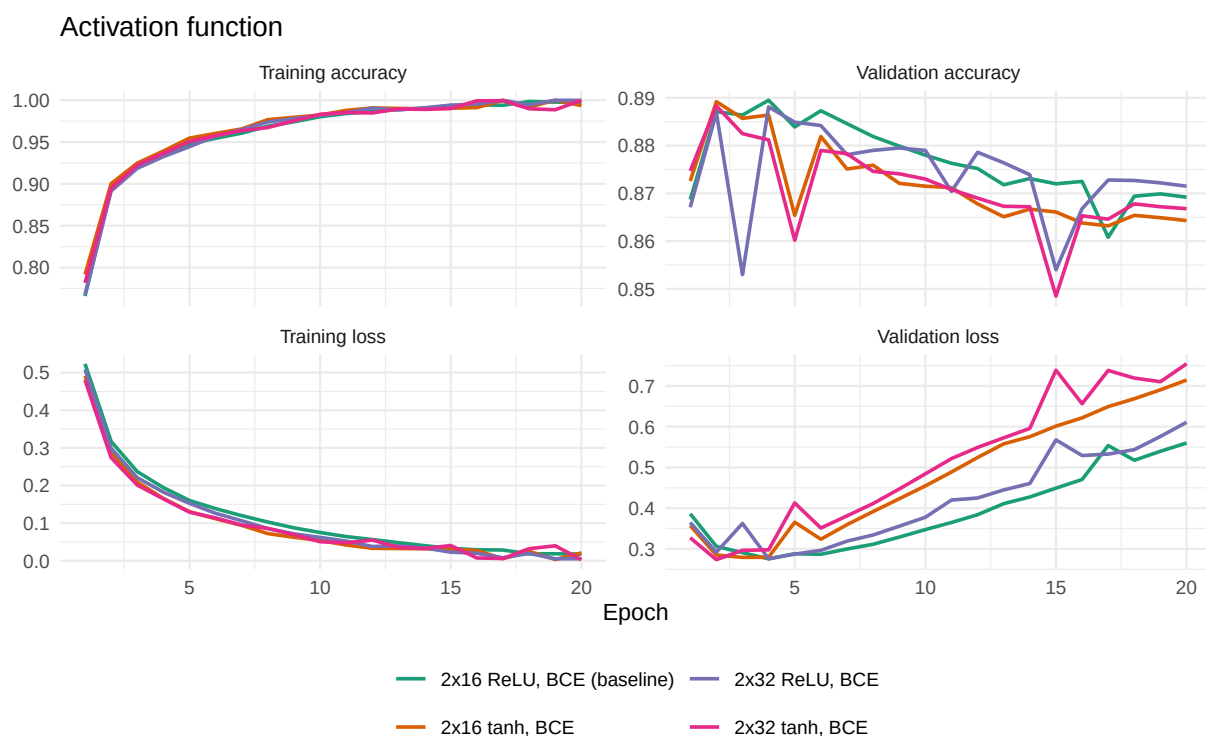


Figure 4: Training and validation curves for ReLU and tanh.

10 Regularization to Improve Validation (Question 5)

Dropout randomly sets 50% of hidden activations to zero during training, and L2 regularization penalizes large weights. Both limit how closely the network can memorize the training reviews.

```
reg_ids <- c("base_2x16", "reg_drop_2x16", "reg_l2_2x16",
            "reg_drop_2x32", "width_2x64", "reg_drop_2x64",
            "reg_drop_l2_2x64")
results_table(reg_ids, "Effect of dropout and L2 regularization")
```

Table 7: Effect of dropout and L2 regularization

Model	Best ep.	Val. (best)	Test (best)	Test (ep. 20)	Gap (ep. 20)
2x16 ReLU, BCE (baseline)	4	88.95%	88.21%	85.84%	12.79%
2x16 ReLU, dropout 0.5	8	89.25%	88.59%	87.64%	8.07%
2x16 ReLU, L2 0.001	4	88.85%	88.27%	86.06%	10.79%
2x32 ReLU, dropout 0.5	5	88.98%	88.40%	87.23%	10.65%
2x64 ReLU, BCE	2	89.26%	88.24%	86.55%	12.55%
2x64 ReLU, dropout 0.5	3	89.02%	88.16%	87.01%	11.69%
2x64 ReLU, dropout 0.5 + L2	3	88.88%	88.14%	87.28%	9.59%

```
plot_curves(reg_ids, "Regularization")
```

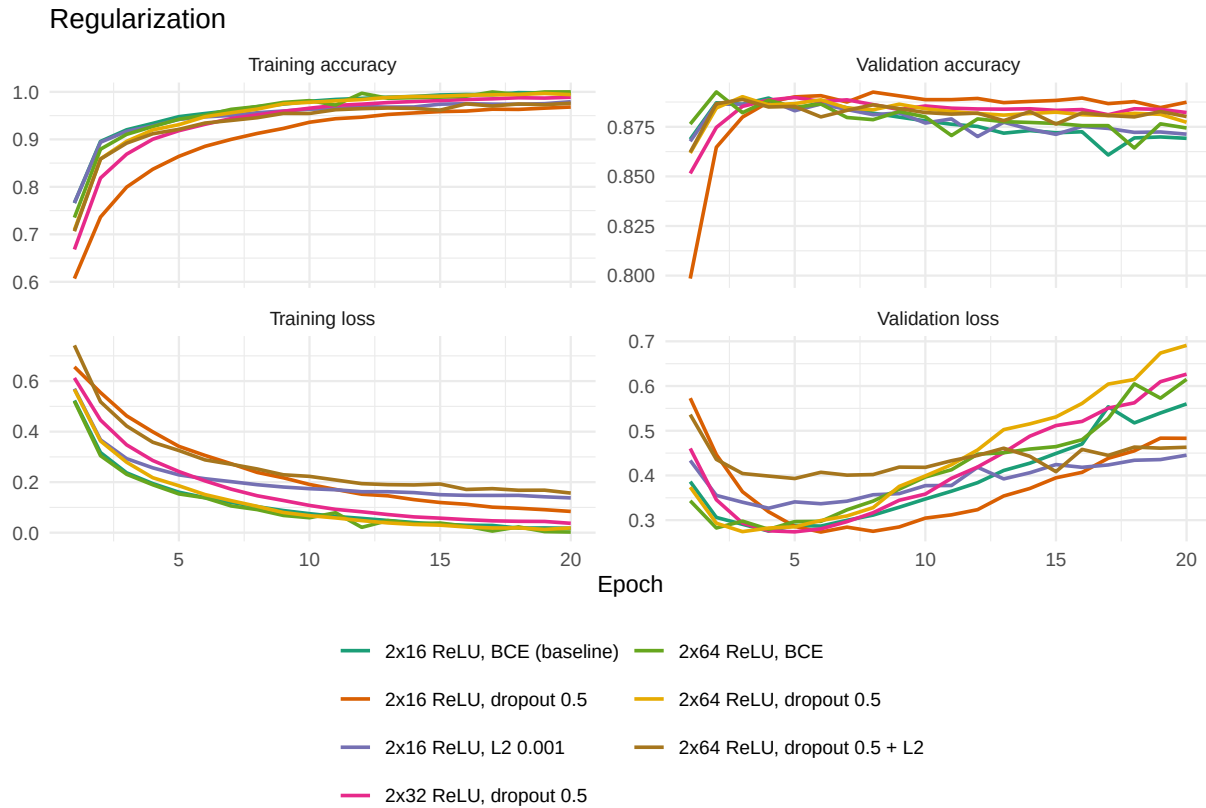


Figure 5: Training and validation curves with and without regularization.

11 Summary of All Experiments

```
results_summary |>
  arrange(desc(best_val_accuracy)) |>
  transmute(
    Rank          = row_number(),
    Model         = label,
    Group        = group,
    `Best epoch`  = best_epoch,
    `Val. acc. (best)` = pct(best_val_accuracy),
    `Test acc. (best)` = pct(test_accuracy_best)
  ) |>
  knitr::kable(caption = "All experiments ranked by best validation accuracy",
    booktabs = TRUE, linesep = "", align = "rllrrr")
```

Table 8: All experiments ranked by best validation accuracy

Rank	Model	Group	Best epoch	Val. acc. (best)	Test acc. (best)
1	2x64 ReLU, BCE	Width	2	89.26%	88.24%
2	2x16 ReLU, dropout 0.5	Regularization	8	89.25%	88.59%
3	1x16 ReLU, BCE	Depth	4	89.04%	88.32%
4	2x8 ReLU, BCE	Width	4	89.03%	88.11%
5	2x64 ReLU, dropout 0.5	Regularization	3	89.02%	88.16%
6	2x16 ReLU, MSE	Loss	4	89.01%	88.34%
7	2x32 ReLU, dropout 0.5	Regularization	5	88.98%	88.40%
8	2x16 ReLU, BCE (baseline)	Baseline	4	88.95%	88.21%
9	2x16 tanh, BCE	Activation	2	88.92%	88.18%
10	2x64 ReLU, dropout 0.5 + L2	Regularization	3	88.88%	88.14%
11	2x16 ReLU, L2 0.001	Regularization	4	88.85%	88.27%
12	2x32 tanh, BCE	Activation	2	88.83%	88.21%
13	2x32 ReLU, BCE	Width	4	88.81%	88.25%
14	3x16 ReLU, BCE	Depth	3	88.78%	88.02%

```

baseline_val <- results_summary$best_val_accuracy[results_summary$id == "base_2x16"]

results_summary |>
  mutate(label = reorder(label, best_val_accuracy)) |>
  pivot_longer(c(best_val_accuracy, test_accuracy_best),
               names_to = "metric", values_to = "accuracy") |>
  mutate(metric = ifelse(metric == "best_val_accuracy",
                        "Validation (best epoch)", "Test (best-epoch weights)")) |>
  ggplot(aes(accuracy, label)) +
  geom_vline(xintercept = baseline_val, linetype = "dashed", colour = "grey50") +
  geom_line(aes(group = label), colour = "grey75") +
  geom_point(aes(colour = group, shape = metric), size = 3) +
  scale_x_continuous(labels = percent_format(accuracy = 0.1)) +
  scale_shape_manual(values = c(17, 16)) +
  scale_colour_brewer(palette = "Dark2") +
  labs(title = "IMDB hyperparameter experiments",
       x = "Accuracy", y = NULL, colour = "Group", shape = NULL) +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom", legend.box = "vertical")

```

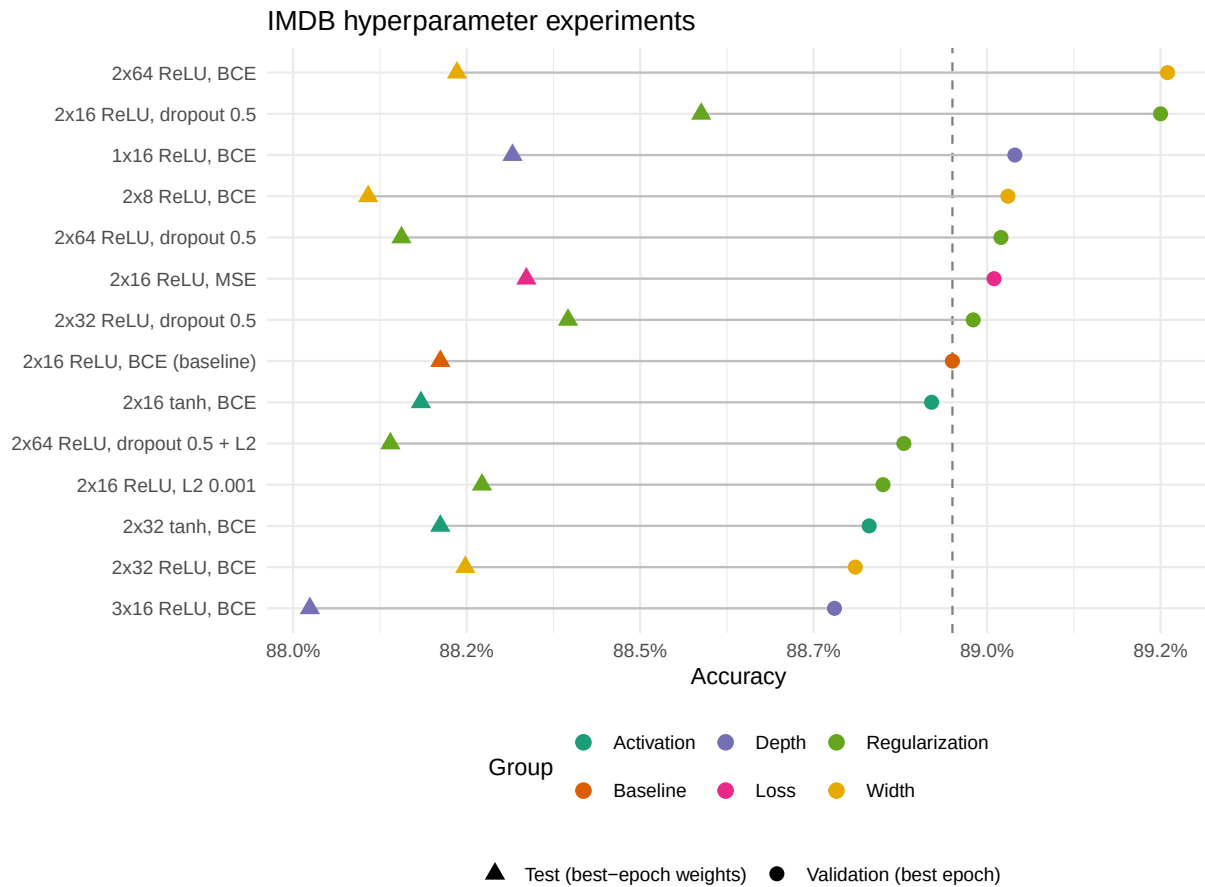


Figure 6: Best validation accuracy and matching test accuracy for every model. The dashed line marks the baseline’s validation accuracy.

12 Final Model

The configuration with the highest validation accuracy is retrained on all 25,000 training reviews for the number of epochs at which it peaked, then evaluated once on the test set.

```
selected <- results_summary |>
  slice_max(best_val_accuracy, n = 1, with_ties = FALSE)
baseline <- results_summary |> filter(id == "base_2x16")

set_random_seed(SEED)
final_model <- build_imdb_model(
  hidden_layers = selected$hidden_layers,
  units         = selected$units,
  activation    = selected$activation,
  loss_function = selected$loss_function,
  dropout_rate  = selected$dropout_rate,
  12            = selected$12
)
```

```

final_history <- final_model |> fit(
  x_train_full, y_train_full,
  epochs      = selected$best_epoch,
  batch_size  = BATCH_SIZE,
  verbose     = 0
)

final_test    <- final_model |> evaluate(x_test, y_test, verbose = 0)
final_test_acc <- as.numeric(final_test[["accuracy"]])

final_results <- tibble(
  Run              = c("Selected model, final",
                      "Baseline, experiment run"),
  Model            = c(selected$label, baseline$label),
  `Training epochs` = c(selected$best_epoch, baseline$best_epoch),
  `Val. acc. (best)` = c(selected$best_val_accuracy,
                        baseline$best_val_accuracy),
  `Test acc.`      = c(final_test_acc, baseline$test_accuracy_best)
)
write_csv(final_results, "results/final_model_results.csv")

final_results |>
  mutate(across(c(`Val. acc. (best)`, `Test acc.`), pct)) |>
  knitr::kable(
    caption = "Final model (retrained on all 25,000 reviews) versus baseline",
    booktabs = TRUE, align = "llrrr")

```

Table 9: Final model (retrained on all 25,000 reviews) versus baseline

Run	Model	Training epochs	Val. acc. (best)	Test acc.
Selected model, final	2x64 ReLU, BCE	2	89.26%	86.62%
Baseline, experiment run	2x16 ReLU, BCE (baseline)	4	88.95%	88.21%

The baseline row shows its test accuracy from the experiment run (trained on 15,000 reviews, best-epoch weights) for reference.

13 Conclusions

- **Hidden layers.** Relative to the two-layer baseline (88.95% validation), one layer gave +0.09 pp (essentially no change (within run-to-run noise)) and three layers gave -0.17 pp (essentially no change (within run-to-run noise)). The best depth was **1x16 ReLU, BCE**. The three-layer network peaked at epoch 3 versus epoch 4 for one layer, so the extra capacity mainly speeds up overfitting.

- **Hidden units.** The best width was **2x64 ReLU, BCE** (89.26%). The epoch-20 overfit gap was 12.77% with 8 units and 12.55% with 64 units.
- **Loss function.** MSE gave +0.06 pp validation accuracy versus binary crossentropy (essentially no change (within run-to-run noise)). Binary crossentropy is the natural loss for a sigmoid output because it penalizes confident wrong predictions much more heavily.
- **Activation.** tanh gave -0.03 pp at 16 units and +0.02 pp at 32 units, relative to ReLU at the same width.
- **Regularization.** The best regularized model (**2x16 ReLU, dropout 0.5**, 89.25%) compares with 89.26% for the best unregularized model (2x64 ReLU, BCE), -0.01 pp. For the 64-unit network, dropout changed the epoch-20 overfit gap from 12.55% to 11.69%, and its test accuracy after 20 epochs was 87.01% versus 86.55% without dropout.
- **Final model.** **2x64 ReLU, BCE** was selected on validation accuracy (89.26%). Retrained on all 25,000 reviews for 2 epoch(s), it reached **86.62%** test accuracy, compared with 88.21% for the baseline.

Overall story. The curves show that the main challenge on this task is overfitting rather than a lack of capacity: the unregularized models reach their best validation accuracy early (between epochs 2 and 4) and then decline while training accuracy keeps rising toward 100%. Across the architecture changes in Questions 1 to 4, best validation accuracy stayed within a 0.5-percentage-point range. The two most useful tools are therefore stopping at the best epoch, which every model here uses for its test score, and regularization such as dropout, which slows the collapse of validation accuracy when a model is trained for longer than it needs.